



Figure 17: Scaling laws for the GPT2 MoE models with 32 experts on the bioS(N) data for **100 exposures**. This figure complements Figure 16 by comparing the effects of varying $topk$ and cap_factor in the 100-exposure insufficiently-trained regime. **Conclusion:** minimal differences are observed across these settings, though deeper models (e.g., GPT2-16-4 with 32 experts) seem easier to train with $topk = cap_factor = 2$.

- For $N = 100K$, we use $wd = 0.01$, $lr = 0.001/0.002/0.005$, and batch size 24 with fp16;
- For $N = 200K$, we use $wd = 0.01$, $lr = 0.001/0.002/0.005$, and batch size 48 with fp16;
- For $N = 500K$, we use $wd = 0.01$, $lr = 0.001/0.002$, and batch size 96 with fp16;
- For $N = 1M$, we use $wd = 0.01$, $lr = 0.0005/0.001/0.002$, and batch size 192 with fp16;
- For $N = 2M$, we use $wd = 0.005$, $lr = 0.0005/0.001$, and batch size 192 with fp16;
- For $N = 5M$, we use $wd = 0.005$, $lr = 0.0003/0.0005/0.001$, and batch size 384 with fp16.

E More on Junk Data vs. Scaling Laws

Recall from Section 10 that our dataset is a mixture, with $1/8$ of the tokens coming from bioS(N) for various N (referred to as “useful data”), and the remaining $7/8$ from “junk data.” We explored three scenarios:

- Junk data being bioS(N') for $N' = 100M$, representing completely random junk;
- Junk data being bioS(N') for $N' = 1K$, representing highly repetitive data; and
- Junk data being bioS(N') for $N' = 100M$, but with a special token appended to the front of each piece of *useful data*.²⁸

For simplicity, within each 512-token context window, we either include only useful data or only junk data (separated by $\langle \text{EOS} \rangle$ tokens). The outcomes are similar when mixing useful and junk data in the same context window. In all three cases, we initially consider a 100-exposure training setting where the useful data receive 100 exposures each during pretraining — thus, the total number of training tokens is approximately 8 times more than in Figure 1(b) (our scaling law for the 100-exposure case without junk data).

In case (A), presenting a *negative result*, we also explore 300-exposure, 600-exposure, and 1000-exposure training settings. Given that the 1000-exposure setting requires 48x more training tokens compared to Figure 1(b), or 4.8x more compared to Figure 1(a), we limited experiments to bioS(N) with $N \leq 200K$ to conserve computational resources. Similarly, for 300-exposure and 600-exposure, we only considered $N \leq 500K$.

In case (B), presenting a *positive result*, we limited our consideration to 100-exposure with $N \leq 1M$.

²⁸This is akin to adding a domain name like wikipedia.org at the beginning of the data; the model lacks prior knowledge that these special token data signify high-quality, useful data. It’s up to the model and the training process to *autonomously* discover this.

In case (C), presenting a *moderately positive result*, we explored both 100-exposure and 300-exposure settings, where, in the 300-exposure setting, we again limited to $N \leq 500K$.

Overall, due to the significantly different training durations (i.e., number of training tokens) across the 100-, 300-, 600-, and 1000-exposure settings, we had to adjust their batch sizes, weight decay, and learning rates accordingly. These adjustments are discussed below.

Parameter 10 (Figure 8). We adhere to the general advice provided in Remark A.2 for selecting parameters in all experiments shown in Figure 8. For negative results (e.g., Figure 8(b), 8(c)), we opted for a smaller batch size to increase the number of trainable steps and explored a wider range of learning rate options. Conversely, for positive results (e.g., Figure 8(f), 8(e)), we sometimes chose a larger batch size to benefit from faster, GPU-accelerated training times and considered a narrower set of learning rate choices. Overall, we have meticulously selected parameters to strengthen negative results as much as possible while intentionally not optimizing positive results to the same extent. This approach ensures a *stronger comparison* and effectively communicates the key message of this section. **Specifically,**

- For Figure 8(b) which is Case (a) of 100-exposure:
 - For $N = 50K$, we use $wd = 0.01$, $lr = 0.0003/0.0005/0.001$, and batch size 12;
 - For $N = 100K$, we use $wd = 0.01$, $lr = 0.0003/0.0005/0.001$, and batch size 24;
 - For $N = 200K$, we use $wd = 0.01$, $lr = 0.0003/0.0005/0.001$, and batch size 48;
 - For $N = 500K$, we use $wd = 0.005$, $lr = 0.00005/0.0001/0.0002/0.0003/0.0005$, and batch size 192;
 - For $N = 1M$, we use $wd = 0.005$, $lr = 0.00005/0.0001/0.0002/0.0003/0.0005$, and batch size 192.
- For Figure 8(c) which is Case (a) of 300-exposure:
 - For $N = 50K$, we use $wd = 0.01$, $lr = 0.0003/0.0005/0.001$, and batch size 96;
 - For $N = 100K$, we use $wd = 0.01$, $lr = 0.0003/0.0005/0.001$, and batch size 192;
 - For $N = 200K$, we use $wd = 0.01$, $lr = 0.0003/0.0005/0.001$, and batch size 192;
 - For $N = 500K$, we use $wd = 0.01$, $lr = 0.0003/0.0005/0.001$, and batch size 192.
- For Figure 8(d) which is Case (a) of 600-exposure:
 - For $N = 50K$, we use $wd = 0.01$, $lr = 0.0003/0.0005/0.001$, and batch size 384;
 - For $N = 100K$, we use $wd = 0.01$, $lr = 0.0003/0.0005/0.001$, and batch size 384;
 - For $N = 200K$, we use $wd = 0.01$, $lr = 0.0003/0.0005/0.001$, and batch size 384;
 - For $N = 500K$, we use $wd = 0.002$, $lr = 0.0003/0.0005/0.001$, and batch size 768.
- For Figure 8(e) which is Case (a) of 1000-exposure:
 - For $N = 50K$, we use $wd = 0.01$, $lr = 0.0005/0.001$, and batch size 384;
 - For $N = 100K$, we use $wd = 0.01$, $lr = 0.0005/0.001$, and batch size 768;
 - For $N = 200K$, we use $wd = 0.01$, $lr = 0.0005/0.001$, and batch size 1536.
- For Figure 8(f) which is Case (b) of 100-exposure:
 - For $N = 50K$, we use $wd = 0.01$, $lr = 0.0003/0.0005$, and batch size 12;
 - For $N = 100K$, we use $wd = 0.01$, $lr = 0.0003/0.0005$, and batch size 24;
 - For $N = 200K$, we use $wd = 0.01$, $lr = 0.0003/0.0005/0.001$, and batch size 96;
 - For $N = 500K$, we use $wd = 0.01$, $lr = 0.0003/0.0005$, and batch size 192;
 - For $N = 1M$, we use $wd = 0.01$, $lr = 0.0003$, and batch size 192.
- For Figure 8(g) which is Case (c) of 100-exposure:
 - For $N = 50K$, we use $wd = 0.01$, $lr = 0.0003/0.0005/0.001$, and batch size 12;
 - For $N = 100K$, we use $wd = 0.01$, $lr = 0.0003/0.0005/0.001$, and batch size 24;
 - For $N = 200K$, we use $wd = 0.01$, $lr = 0.0002/0.0003/0.0005/0.001$, and batch size 96;
 - For $N = 500K$, we use $wd = 0.005$, $lr = 0.0002/0.0003/0.0005$, and batch size 192;
 - For $N = 1M$, we use $wd = 0.005$, $lr = 0.0002/0.0003/0.0005$, and batch size 192.
- For Figure 8(h) which is Case (c) of 300-exposure:
 - For $N = 50K$, we use $wd = 0.01$, $lr = 0.0003/0.0005/0.001$, and batch size 96;
 - For $N = 100K$, we use $wd = 0.01$, $lr = 0.0003/0.0005/0.001$, and batch size 192;

- For $N = 200K$, we use $wd = 0.01$, $lr = 0.0003/0.0005/0.001$, and batch size 192;
- For $N = 500K$, we use $wd = 0.005$, $lr = 0.0003/0.0005/0.001$, and batch size 384.

F Proof of Theorem 3.2

We present a crucial lemma that establishes the bit complexity required to encode random variables based on the probability that these variables match specific reference values.

Lemma F.1. *Let $\mathcal{Q}_1, \dots, \mathcal{Q}_k$ be fixed sets (we call **domains**), and assume that for each $i \in [k]$, Q_i is independently and randomly chosen from its corresponding domain $\mathcal{Q}_i$. Denote $Q = (Q_1, \dots, Q_k)$ and view Q as the **training data**.*

*Assume there exists a function $W(Q) \in \mathcal{W}$, which we regard as the **parameters** of a model computed (i.e., trained) from the training data Q .*

*Furthermore, consider an **evaluation function** F_i that predicts*

$$\forall i \in [k]: \quad P_i = F_i(W(Q), Q_1, Q_2, \dots, Q_{i-1}, R) \quad \text{with} \quad p_i(Q) \stackrel{\text{def}}{=} \Pr_R[P_i = Q_i \mid Q] .$$

Here, F is parameterized by $W(Q)$ and may rely on previous data $Q_1, \dots, Q_{i-1}$, and new randomness R . Then, it follows that

$$\log |\mathcal{W}| \geq \sum_{i \in [k]} \log \left(\mathbb{E}_Q [p_i(Q)] \times |\mathcal{Q}_i| \right) \geq \mathbb{E}_Q \left[\sum_{i \in [k]} \log (p_i(Q) \times |\mathcal{Q}_i|) \right] . \quad (\text{F.1})$$

Proof of Lemma F.1. Since the second inequality of (F.1) trivially comes from Jensen’s inequality, we only prove the first one.

When $i = 1$, we have $P_1 = F_1(W(Q), R)$ and one can prove the lemma by a simple counting argument, using the property that $\forall R, P_1 = F_1(W(Q), R)$ has at most $|\mathcal{W}|$ choices of values.

When $i \geq 2$, we can merge data points Q_1, Q_2 to be a new data point Q' with domain $\mathcal{Q}' = \mathcal{Q}_1 \times \mathcal{Q}_2$. We can construct $P' = (P_1, P_2)$ from function F_1, F_2 by sampling R_1 to generate $P_1 = F_1(W(Q), R)$, and then sample independent R_2 to generate $P_2 = F_2(W(Q), Q_1, R)$. We know that $\Pr_{R_1, R_2}[P' = Q' \mid Q] = \Pr_{R_1}[P_1 = Q_1 \mid Q] \Pr_{R_2}[P_2 = Q_2 \mid Q] = p_1(Q) \cdot p_2(Q)$. The lemma now follows using the following identity:

$$\log(p_1(Q)|\mathcal{Q}_1|) + \log(p_2(Q)|\mathcal{Q}_2|) = \log(p_1(Q)p_2(Q)|\mathcal{Q}_1||\mathcal{Q}_2|) . \quad \square$$

F.1 Warmup Examples

Let us first see two warmup applications of Lemma F.1.

Value-only. Let $g_1, \dots, g_N \in [T]$, where each g_i is i.i.d. uniformly chosen at random from $[T]$. Think of these as *values*. Suppose a model, parameterized by W , is trained on the training data $\mathcal{Z} = (g_1, \dots, g_N)$. Assume this model, for a given index $i \in [N]$, can generate a random prediction f_i corresponding to g_i . We can represent this model as $f_i(W(\mathcal{Z}), R)$, where R denotes the randomness. The cross-entropy loss for this scenario (averaged over all possible training data) is expressed as

$$\mathbf{loss} \stackrel{\text{def}}{=} \mathbb{E}_g [\mathbf{loss}(g)] \stackrel{\text{def}}{=} \mathbb{E}_g \left[\frac{1}{N} \sum_{i \in [N]} -\log \Pr_{f_i}[f_i = g_i] \right] \geq 0$$

Now we apply Lemma F.1 by setting $\mathcal{Q}_1 = \dots \mathcal{Q}_N = [T]$, $Q_i = g_i$, and $P_i = f_i$. We have

$$\log |\mathcal{W}| \geq \mathbb{E}_g \left[\sum_{i \in [N]} \log \Pr_{f_i}[f_i = g_i] + \log T \right] = N \log T - N \mathbb{E}_g \mathbf{loss}(g) = \mathbb{E}_g N \log \frac{T}{e^{\mathbf{loss}(g)}} .$$

Changing the base immediately yields a bit complexity lower bound of $\log_2 |\mathcal{W}| \geq N \log_2 \frac{T}{e^{\text{loss}}}$. As the loss approaches zero, this matches the bit complexity upper bound.

Name-only. Let $g_1, \dots, g_N \in [N_0]$ be N distinct elements from $[N_0]$, sampled uniformly at random without replacement, and considered as *names*. Suppose a model f , parameterized by W , is trained on the dataset $\mathcal{Z} = (g_1, \dots, g_N)$ to predict a name. We denote this as $f(W(\mathcal{Z}), R)$, where R represents randomness. The cross-entropy loss for this scenario is defined as

$$\text{loss} \stackrel{\text{def}}{=} \mathbb{E}_g[\text{loss}(g)] \stackrel{\text{def}}{=} \mathbb{E}_g \left[\frac{1}{N} \sum_{i \in [N]} -\log \Pr_f[f = g_i] \right] \geq 0$$

To apply Lemma F.1, we define $\mathcal{Q}_1 = [N_0]$, $\mathcal{Q}_2 = [N_0 - 1]$, and continue until $\mathcal{Q}_N = [N_0 - N + 1]$. After uniformly randomly generating $Q_1, \dots, Q_N$ from $\mathcal{Q}_1, \dots, \mathcal{Q}_N$, we construct $(g_1, \dots, g_N) \in [N]^N$ as follows: set $g_1 = Q_1$; for g_2 , set it to Q_2 if $Q_2 < Q_1$, otherwise $g_2 = Q_2 + 1$; and in general, define g_i as the Q_i -th smallest element in $[N_0] \setminus \{g_1, \dots, g_{i-1}\}$. This method provides an alternative way to generate $\mathcal{Z} = (g_1, \dots, g_N)$, denoted as $\mathcal{Z}(Q)$. For each $i \in [N]$, we define P_i as follows: first, generate $f = f(W(\mathcal{Z}), R_i)$ using fresh randomness R_i . Set $P_i \stackrel{\text{def}}{=} s$ if f is the s -th smallest element in $[N_0] \setminus \{g_1, \dots, g_{i-1}\}$, or a special symbol such as $\emptyset$ if f is among $\{g_1, \dots, g_{i-1}\}$. (**Note importantly**, this definition of P_i necessitates knowledge of $g_1, \dots, g_{i-1}$; however, this is permissible as Lemma F.1 allows P_i to depend on $Q_1, \dots, Q_{i-1}$.) For every fixed Q (and thus fixed g),

$$\sum_{i \in [N]} \log(p_i(Q)) = \sum_{i \in [N]} \log(\Pr_{P_i}[P_i = Q_i]) = \sum_{i \in [N]} \log(\Pr_{R_i}[f(W(\mathcal{Z}), R_i) = g_i]) = -N \text{loss}(g)$$

Applying Lemma F.1 we have

$$\log |\mathcal{W}| \geq \mathbb{E}_Q \left[\sum_{i \in [N]} \log(p_i(Q) \times |N_0 - i + 1|) \right] \geq \mathbb{E}_Q \left[N \log \frac{N_0 - N}{e^{\text{loss}(g)}} \right] = N \log \frac{N_0 - N}{e^{\mathbb{E}_Q \text{loss}(g)}}.$$

Ideally, if the model f can perfectly memorize the entire training set $\{Z\} = (g_1, \dots, g_N)$, its best possible loss $\text{loss}(g) = \log N$ is achieved. Thus, if the model can perfectly learn this training set, the bit complexity lower bound satisfies $\log |\mathcal{W}| \geq N \log \frac{N_0 - N}{N} \geq (1 - o(1))N \log \frac{N_0}{N}$ when $N \ll N_0$.

F.2 Main Proof

We recommend that readers first review the warmup examples in Section F.1 before proceeding with this proof.

Proof of Theorem 3.2. Let us first construct the domains $\mathcal{Q}_i$'s in Lemma F.1.

1. Let $\mathcal{Q}_1 = [N_0]$, $\mathcal{Q}_2 = [N_0 - 1] \dots \mathcal{Q}_N = [N_0 - N + 1]$.
2. Let $(\mathcal{Q}_{N+jD+1}, \dots, \mathcal{Q}_{N+jD+D}) = ([T^L], [T^L - 1], \dots, [T^L - D + 1])$ for every $j = 0, \dots, K - 1$.
3. Let $\mathcal{Q}_{N+KD+1} = \dots = \mathcal{Q}_{N+KD+NK} = [D^C]$.

Recall that each Q_i is independently and uniformly generated at random from $\mathcal{Q}_i$. We now present an alternative method for generating the training dataset $\mathcal{Z}(Q)$.

1. Construct $\mathcal{N} = (n_1, \dots, n_N)$ as follows: Let n_1 be the Q_1 -th name from $\mathcal{N}_0$; for $i > 1$, let n_i be the Q_i -th name from $\mathcal{N}_0 \setminus \{n_1, \dots, n_{i-1}\}$.
2. For each $a' \in [K]$, let a be the a' -th attribute in $\mathcal{A}$. Construct $\mathcal{D}_a = (w_1, \dots, w_D)$ as follows: Let w_1 be the $Q_{N+(a'-1)D+1}$ -th element in $\mathcal{T}^L$; for $i > 1$, let w_i be the $Q_{N+(a'-1)D+i}$ -th element in $\mathcal{T}^L \setminus \{w_1, \dots, w_{i-1}\}$.